Informe de implementación y resultados

Integrantes

Juan Jose Rodriguez Lozano - 202419084

Steven Fernando Aragon Alvarez - 202418804

Yoel Steven Montoya - …

Proyecto Integrador
Parcial I

Profesor

Manuel Alejandro Pastrana Pardo, PhD

Universidad Del Valle

Facultad De Ingeniería

Santiago de Cali

2026

Link del repositorio: https://github.com/TofiTurtle/Parcial_Infraestructuras_I.git

1. Introducción

Este informe documenta la implementación del Juego de la Vida de John Conway en dos versiones, una secuencial y una paralela, y compara su rendimiento. El objetivo es aplicar los conceptos de paralelismo vistos en clase y medir con datos cuánta aceleración se obtiene al repartir el trabajo entre varios procesos.

El programa está escrito en Python 3.14 y usa Pygame (pygame-ce 2.5.8) para la interfaz gráfica. La versión paralela usa MPI (Message Passing Interface) a través de la biblioteca mpi4py, con Microsoft MPI (MS-MPI) como implementación en Windows. Se entregan dos archivos: juego_vida.py, que contiene el juego con interfaz gráfica y permite elegir entre modo secuencial y paralelo (se lanza con mpiexec -n p python juego_vida.py), y benchmark.py, que mide tiempos sin interfaz, verifica la correctitud y genera las gráficas (se lanza con python benchmark.py y él mismo invoca a mpiexec en cada medición).

2. Descripción del juego

La población se representa con una matriz cuadrada de n x n, donde 1 es una célula viva y 0 una célula muerta. En cada generación todas las células se actualizan al mismo tiempo según estas reglas:

- Una célula viva con menos de 2 vecinos vivos muere por soledad.
- Una célula viva con 2 o 3 vecinos vivos sobrevive.
- Una célula viva con más de 3 vecinos vivos muere por sobrepoblación.
- Una célula muerta con exactamente 3 vecinos vivos nace.

Decisiones de diseño: las celdas fuera de la matriz se consideran muertas, y el estado inicial es aleatorio con una probabilidad de 50 % de estar viva.

3. Implementación secuencial

La versión secuencial es el punto de referencia para medir el speedup. Está compuesta por estas funciones:

- generar_matriz_aleatoria(dim): crea la matriz inicial.
- conteo_vecinos(m, i, j): recorre los 8 vecinos de una celda, verificando que estén dentro de la matriz, y cuenta los vivos.

- calcular_filas(m, inicio, fin): aplica las reglas a las filas del rango [inicio, fin) y devuelve esas filas ya calculadas. No modifica la matriz m, solo la lee.
- siguiente_generacion_seq(m): calcula la generación completa llamando a calcular_filas(m, 0, n).

La función calcular_filas la usan tanto la versión secuencial como la paralela. Así, cualquier diferencia de tiempo se debe únicamente al reparto del trabajo y no a lógicas distintas.

La versión inicial del código usaba dos funciones (survives_seq y revives_seq) y una matriz auxiliar m_aux. Funcionaba, pero dependía de una condición implícita: m_aux debía ser una copia exacta de m al empezar cada generación, porque solo se escribían las celdas que cambiaban. Si esa condición se rompía, el resultado quedaba mal sin ningún error visible. Además, la matriz auxiliar es un estado compartido que complica el reparto entre procesos. Por eso se reemplazó por una función que construye las filas nuevas desde cero.

La interfaz gráfica corre únicamente en el proceso 0 (rank 0): Espacio o Enter avanzan una generación (mantenerlos presionados avanza de forma continua) y el título de la ventana muestra el modo, el número de procesos, la generación y el tiempo del último cálculo.

4. Estrategia de paralelización

4.1 Tecnología elegida: MPI con mpi4py

Se eligió MPI mediante mpi4py, que lanza varios procesos del sistema operativo con mpiexec -n p y los comunica por paso de mensajes. El cálculo es intensivo en CPU y está implementado en Python puro. En la implementación estándar de CPython, el GIL (Global Interpreter Lock) permite que solo un hilo ejecute código Python a la vez, por lo que con threading no se esperaría una aceleración. En cambio, con procesos, cada uno tiene su propio intérprete y su propio GIL, y pueden ejecutarse en núcleos distintos. Este razonamiento es tanto teórico como empírico, ya que se basa en la experiencia de los trabajos realizados anteriormente.

Todos los procesos ejecutan el mismo programa (modelo SPMD) y se distinguen por su identificador: rank (de 0 a p-1) y size (número total de procesos). El proceso 0 maneja la interfaz y reparte el trabajo; los demás, mediante la función bucle_trabajador, esperan una matriz, calculan su bloque y lo devuelven. El proceso 0 también calcula un bloque, de modo que con p procesos trabajan p procesos y no p-1.

4.2 División del trabajo

La matriz se divide en bloques de filas consecutivas, uno por proceso. La función dividir_filas(n, p) devuelve la lista de rangos (inicio, fin) y cada proceso toma el que corresponde a su rank. Si n no es divisible entre p, los primeros n mod p bloques reciben una fila extra. Por ejemplo, dividir_filas(10, 3) da [(0, 4), (4, 7), (7, 10)].

Se eligió dividir por filas porque la matriz es una lista de filas: un bloque de filas es un trozo contiguo fácil de calcular, enviar y recibir, y reconstruir la matriz se reduce a concatenar los bloques en orden. Dividir por columnas o por bloques 2D obligaría a ensamblar piezas no contiguas.

El balance de carga es bueno sin esfuerzo adicional: el costo por celda es prácticamente constante (siempre se revisan los 8 vecinos, esté viva o no la célula), así que bloques del mismo tamaño tardan casi lo mismo y no hace falta un reparto dinámico.

4.3 Sincronización

- **Sin locks ni mutexes.** Cada proceso solo lee la generación actual (que recibió por bcast) y devuelve su bloque. Los procesos no comparten memoria, así que no hay condiciones de carrera.
- **Barrera.** comm.gather no devuelve en el proceso 0 hasta que llegaron los bloques de todos los procesos. Esto funciona como una barrera entre generaciones: la generación t+1 no empieza hasta que la t está completa y ensamblada. Los procesos trabajadores, por su parte, quedan bloqueados en bcast hasta que el proceso 0 les envía la siguiente matriz.
- **Orden.** gather entrega los bloques ordenados por rank, así que cada bloque queda en su posición correcta al concatenar.
- **Terminación.** El proceso 0 envía una matriz None con bcast como orden de terminar; los trabajadores salen de su bucle. Se hace dentro de un finally, para que los demás procesos no queden bloqueados aunque el proceso 0 falle.

4.4 Comunicación

Los procesos no comparten memoria y se comunican con operaciones colectivas de mpi4py. En cada generación el proceso 0 envía la matriz completa a todos los procesos con comm.bcast y recibe de vuelta, con comm.gather, el bloque calculado por cada uno. Con las funciones en minúscula, mpi4py serializa los objetos de Python (pickle) antes de enviarlos, así que la matriz, una lista de listas, se serializa y se deserializa en cada generación.

Las filas en el borde de un bloque necesitan las filas vecinas del bloque contiguo. Nuestra implementación lo resuelve de la forma más simple: cada proceso recibe la matriz completa

y por eso puede ver todos sus vecinos. Es correcto y fácil de entender, pero cada proceso recibe y deserializa n^2 celdas aunque solo calcule n^2/p. En un esquema de MPI más fino, lo habitual sería repartir con scatter solo el bloque de cada proceso e intercambiar con sus vecinos una fila de borde por arriba y una por abajo (filas halo). Esto se plantea como mejora en la sección 10.

4.5 Flujo de una generación

1. El proceso 0 envía la matriz actual a todos con bcast (los demás están esperando en bucle_trabajador).
2. Cada proceso obtiene su rango de filas con dividir_filas(n, size)[rank].
3. Cada proceso ejecuta calcular_filas sobre su rango y obtiene su bloque.
4. Todos los bloques se reúnen en el proceso 0 con gather, que además funciona como barrera.
5. El proceso 0 concatena los bloques en orden y obtiene la nueva matriz.
6. La interfaz convierte la matriz en imagen y la dibuja.

Los procesos MPI se crean una sola vez, al lanzar el programa con mpiexec, y se reutilizan en todas las generaciones, para no pagar la creación de procesos en cada paso.

5. Desafíos encontrados

- **El GIL.** Con Python puro, los hilos no dan aceleración en un cálculo intensivo de CPU. Esto condicionó la elección de procesos en lugar de hilos.
- **Memoria distribuida.** Con MPI, los procesos no comparten memoria: todo dato que va y viene se serializa y se copia. Es la principal fuente de sobrecosto y limita el speedup.
- **Condiciones de carrera.** Si varios procesos escribieran sobre una misma matriz mientras otros la leen, una celda podría ver vecinos ya actualizados de la generación nueva y el resultado sería incorrecto. Se evitó por diseño: la matriz actual es de solo lectura y cada proceso devuelve su propio bloque.
- **Matriz auxiliar con condición implícita.** La versión inicial dependía de que m_aux fuera igual a m al empezar cada generación. Se eliminó para que el resultado no dependiera del contenido previo de ningún buffer.
- **Sincronización entre generaciones y terminación.** Se necesita que todos los bloques estén listos antes de pasar a la siguiente generación, y que ningún proceso quede esperando para siempre (interbloqueo). Se resolvió con bcast y gather como puntos de sincronización, y con una orden de terminar enviada en un finally.

- **Bordes entre bloques.** Las filas de borde dependen de celdas que pertenecen a otro bloque. Se resolvió enviando la matriz completa a cada proceso, a costa de más comunicación.
- **El número de procesos se fija al lanzar.** En MPI el valor de p lo da mpiexec -n p, así que cada medición con p distinto es un lanzamiento nuevo. El benchmark actúa de “director”: lanza mpiexec con subprocess por cada prueba y el proceso 0 le devuelve el tiempo medido en un archivo temporal. El arranque de los procesos no se mide y, además, se ejecuta una generación de calentamiento que tampoco se cuenta.
- **Configuración en Windows.** El ejecutable mpiexec de MS-MPI no siempre está en el PATH, por lo que el benchmark lo busca en las carpetas de instalación habituales antes de fallar con un mensaje claro.

6. Metodología de medición

- **Equipo:** cpu: 12th Gen Intel(R) Core(TM) i7-12650HX (2.00 GHz), memoria RAM: 16,0 GB (15,7 GB utilizables) y sistema operativo Windows 11. El sistema reporta 20 hilos lógicos (os.cpu_count()). Python 3.14.6, mpi4py y Microsoft MPI.
- **Tamaños de matriz:** 100x100, 500x500 y 1000x1000.
- **Número de procesos:** 1, 2, 4, 8, 16, 20 y 40. Son las potencias de 2 por debajo de los hilos lógicos, el valor exacto (20) y el doble (40), para ver qué ocurre con más procesos que hilos. El caso de 1 proceso sirve de control: mide el sobrecosto del mecanismo paralelo (bcast y gather) sin repartir trabajo. Cada valor es un lanzamiento distinto con mpiexec -n p.
- **Tiempo medido:** promedio por generación sobre 20 generaciones (100x100), 5 (500x500) y 3 (1000x1000). Se usan más repeticiones donde cada generación dura menos. El tiempo lo mide el proceso 0 con time.perf_counter alrededor del bucle de generaciones, de modo que incluye el envío de la matriz, el cálculo, la recolección y la concatenación. El benchmark completo se ejecutó tres veces; las tablas y gráficas de la sección 7 muestran la mediana de las tres ejecuciones y la sección 8.3 las compara.
- **Calentamiento:** antes de medir cada configuración paralela se ejecuta una generación que no se cuenta. El arranque de los procesos con mpiexec tampoco se mide.
- **Matriz inicial:** generada con semilla 42, para que todas las configuraciones partan del mismo estado.

- **Speedup:** tiempo secuencial dividido entre tiempo paralelo. La versión secuencial se mide con el mismo mecanismo (un lanzamiento con 1 proceso, sin bcast ni gather).
- **Correctitud:** antes de medir, el benchmark ejecuta 10 generaciones de una matriz de 60x60 con semilla 42, en secuencial y en paralelo con 4 procesos (mpiexec -n 4), y el proceso 0 comprueba que las matrices resultantes sean idénticas. En las tres ejecuciones se obtuvo el mensaje de verificación correcta.

7. Resultados

Tiempo por generación (segundos)

Procesos	100x100	500x500	1000x1000
Secuencial	0.010	0.284	1.159
1	0.011	0.292	1.187
2	0.006	0.153	0.646
4	0.003	0.086	0.354
8	0.003	0.067	0.268
16	0.002	0.050	0.213
20	0.003	0.057	0.216
40	0.007	0.066	0.307

Speedup (tiempo secuencial / tiempo paralelo)

Procesos	100x100	500x500	1000x1000
1	0.93	0.98	0.98
2	1.68	1.85	1.80
4	3.08	3.31	3.27
8	3.20	4.24	4.33
16	4.52	5.65	5.43

20	2.80	4.98	5.38
40	1.39	4.30	3.77

Medianas de tres ejecuciones del benchmark (20, 5 y 3 generaciones por prueba en 100x100, 500x500 y 1000x1000). Los tiempos se muestran con tres decimales, que es como los imprime el benchmark. El speedup es la mediana de los speedups de cada ejecución, calculados con los tiempos sin redondear, por lo que no siempre coincide con dividir los valores de la tabla de tiempos. La sección 8.3 compara las tres ejecuciones.

Gráficas

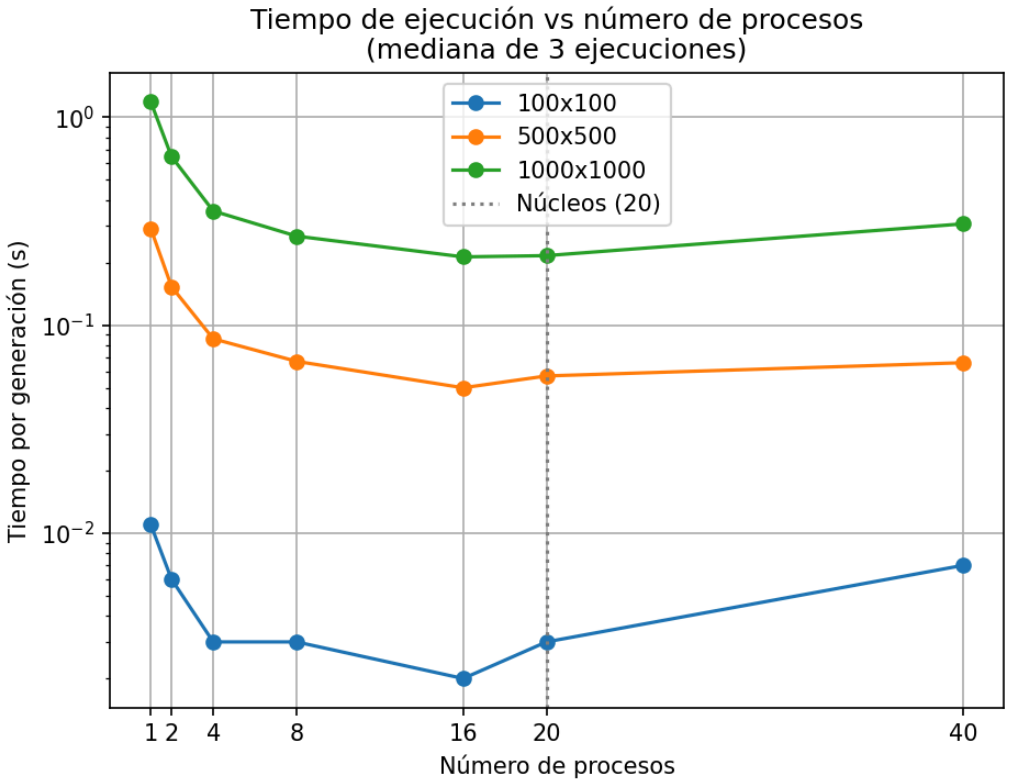


Figura 1. Tiempo de ejecución por generación vs. número de procesos (escala logarítmica, mediana de tres ejecuciones).

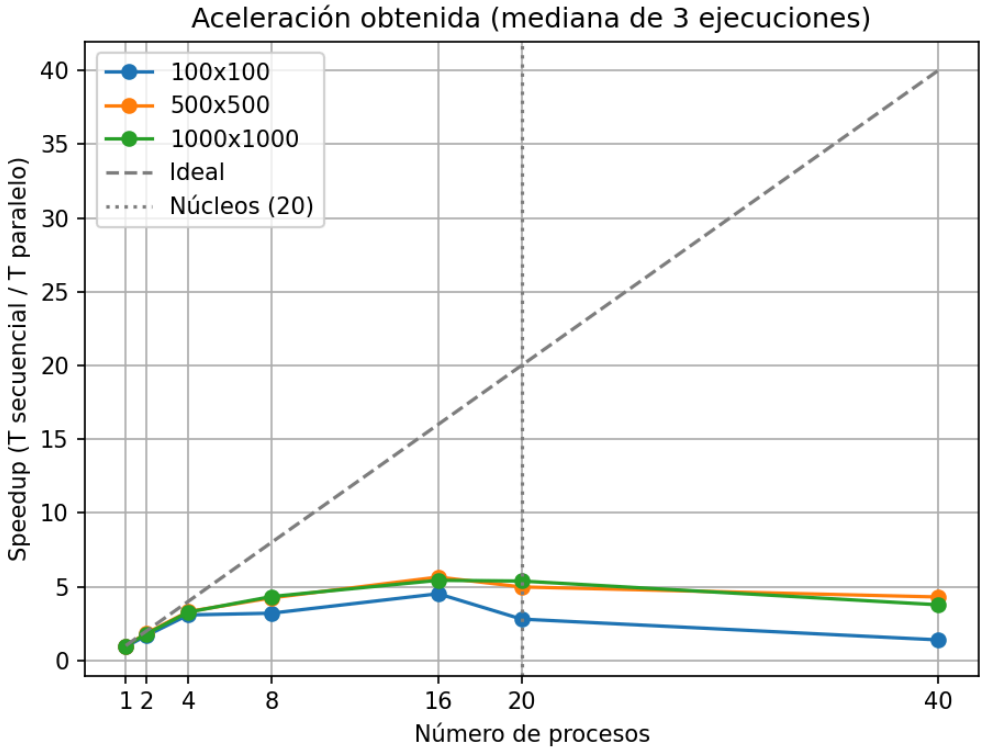


Figura 2. Aceleración (speedup) vs. número de procesos, con la línea ideal y los hilos lógicos del equipo (mediana de tres ejecuciones).

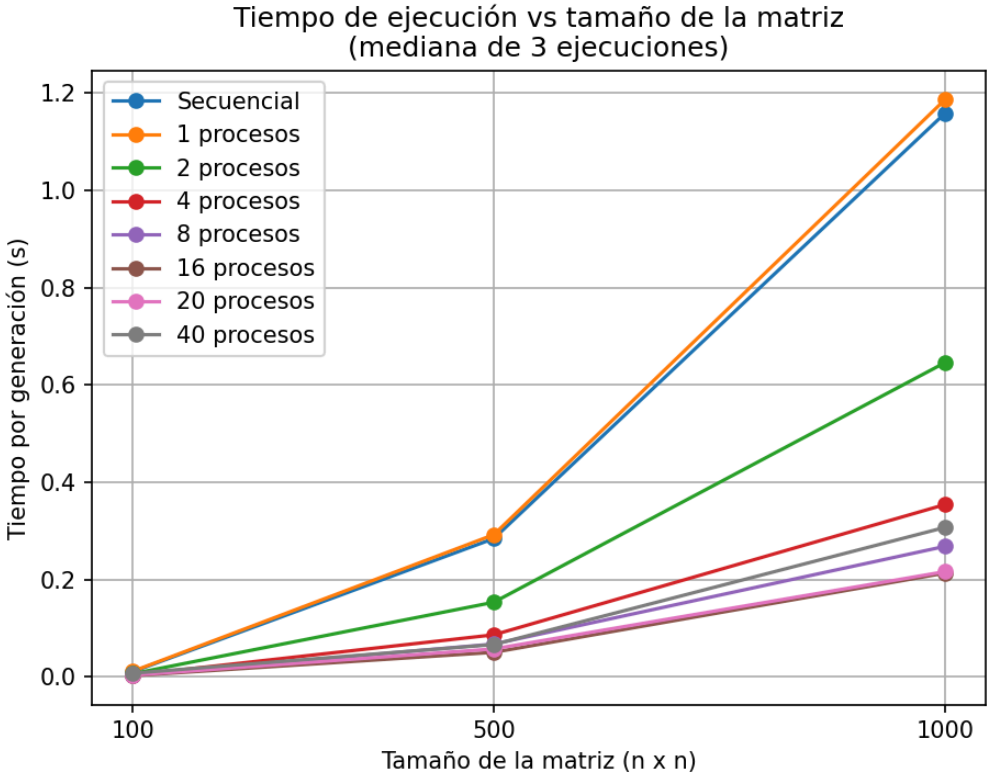


Figura 3. Tiempo de ejecución por generación vs. tamaño de la matriz (mediana de tres ejecuciones).

8. Análisis de rendimiento

8.1 Qué muestran los datos

- **Un proceso equivale al secuencial** (speedup entre 0.93 y 0.98 en la mediana, y entre 0.88 y 1.02 considerando las tres ejecuciones). El sobrecosto fijo de bcast y gather con un solo proceso es pequeño: entre 2 % y 8 %, y en 100x100 equivale a apenas 1 ms, dentro del ruido de la medición.
- **500x500 y 1000x1000 se comportan casi igual.** Con 2 procesos el speedup es 1.85 y 1.80, con 4 es 3.31 y 3.27, con 8 llega a 4.24 y 4.33 y con 16 alcanza el máximo: 5.65 y 5.43. Con 20 procesos queda en 4.98 y 5.38 y con 40 baja a 4.30 y 3.77. En 1000x1000, 16 y 20 procesos no se distinguen (5.43 y 5.38; en la ejecución 2 incluso 20 superó a 16, con 5.52 contra 5.33).
- **100x100.** El máximo es 4.52 con 16 procesos, con una generación de solo 2 ms. Con 20 baja a 2.80 y con 40 a 1.39, aunque en las tres ejecuciones sigue siendo más rápido que el secuencial (entre 1.28 y 1.48). Una generación dura unos 10 ms, así que el costo de coordinar procesos es comparable al del cálculo. Además, es el tamaño menos estable entre ejecuciones (sección 8.3).
- **El tamaño importa hasta cierto punto.** Con 8 procesos el speedup es 3.20, 4.24 y 4.33 para 100x100, 500x500 y 1000x1000, y con 16 es 4.52, 5.65 y 5.43. De 100 a 500 la ganancia es clara, pero de 500 a 1000 ya no mejora (con 16 procesos, 500x500 es incluso un poco mayor). Es coherente con que tanto el cálculo como el envío de la matriz crezcan con n^2: pasado cierto tamaño, la proporción entre ambos deja de cambiar.
- **Más procesos que hilos lógicos empeora** en los tres tamaños respecto a 16 procesos: con 40 procesos el speedup cae a 1.39, 4.30 y 3.77. Aun así, ninguno queda por debajo del secuencial.

8.2 Por qué el speedup se aleja del ideal

1. **Comunicación y serialización.** En cada generación el proceso 0 serializa la matriz completa y la envía a todos; cada proceso la deserializa entera (n^2 celdas) aunque solo calcule n^2/p, y el proceso 0 además deserializa los p bloques recibidos. Parte de este trabajo no disminuye al aumentar p. Los datos son coherentes con esto: si se despeja la ley de Amdahl con los speedups medianos

de 1000x1000, la fracción efectiva no paralelizable resulta de aproximadamente 11 % con 2 procesos, 7 % con 4, 12 % con 8, 13 % con 16, 14 % con 20 y 25 % con 40 (en 500x500 resulta entre 7 % y 16 % hasta 20 procesos y 21 % con 40). Es decir, cerca de un 10 % a 15 % del tiempo no se paraleliza, lo que limita el speedup máximo teórico a un valor del orden de 8. Esta es la explicación más consistente con los datos, pero no se midió directamente el tiempo de serialización ni el de comunicación.

2. **Parte secuencial fija.** Concatenar los bloques en el proceso 0 y, en la interfaz, convertir la matriz en imagen se ejecutan en un solo proceso (ley de Amdahl).
3. **Hardware.** Los 20 hilos lógicos no equivalen a 20 núcleos completos iguales: el procesador es híbrido (núcleos de alto rendimiento y de eficiencia) y los hilos de un mismo núcleo comparten recursos. Además, los procesos compiten por caché y memoria, y el rendimiento por proceso baja al aumentar su número. La eficiencia (speedup dividido entre procesos) cae de 82 % a 83 % con 4 procesos a 34 % a 35 % con 16 en 500x500 y 1000x1000.
4. **Más procesos que hilos.** Con 40 procesos sobre 20 hilos, el sistema operativo debe alternar procesos y hay cambios de contexto; es posible que los procesos que esperan en operaciones de MPI también consuman CPU mientras esperan, pero no se verificó. Por eso con 40 procesos los tiempos empeoran en los tres tamaños.
5. **Bloques demasiado pequeños.** Con 100x100 y 40 procesos cada bloque tiene apenas 2 o 3 filas, y la coordinación domina sobre el cálculo; incluso con 16 procesos son unas 6 filas por bloque y el tiempo ya es de solo 2 ms.

8.3 Fiabilidad de las mediciones

El benchmark se ejecutó tres veces en el mismo equipo y con la misma configuración. La tabla compara las tres ejecuciones en algunas mediciones representativas; las tablas de la sección 7 muestran la mediana. Cada tiempo se imprime con tres decimales (±0.5 ms al leerlo), aunque el benchmark lo guarda completo y calcula el speedup con los valores sin redondear.

Medición	Ejecución 1	Ejecución 2	Ejecución 3
Secuencial 500x500 (s)	0.284	0.284	0.299
Secuencial 1000x1000 (s)	1.159	1.149	1.164

Speedup con 4 procesos, 100x100	3.26	3.08	1.91
Speedup con 8 procesos, 500x500	4.10	4.24	4.64
Speedup con 16 procesos, 500x500	5.48	5.65	6.01
Speedup con 4 procesos, 1000x1000	3.27	3.23	3.29
Speedup con 16 procesos, 1000x1000	5.43	5.33	5.47
Speedup con 20 procesos, 1000x1000	5.38	5.52	5.28
Speedup con 40 procesos, 1000x1000	3.77	3.47	3.87

Las matrices grandes son muy estables: en 1000x1000 el secuencial varía entre 1.149 s y 1.164 s (1.3 %), el speedup con 4 procesos entre 3.23 y 3.29 y con 16 procesos entre 5.33 y 5.47. En 500x500 la variación es mayor: el secuencial va de 0.284 s a 0.299 s, el speedup con 4 procesos de 2.98 a 3.52 y con 16 procesos de 5.48 a 6.01. Ninguna ejecución mostró un valor imposible, como un speedup mayor que el número de procesos (con 2 procesos el máximo fue 1.97); con 1 proceso el speedup va de 0.88 a 1.02, lo que da una idea del ruido de la medición.

El tamaño 100x100 es el menos estable. Con 4 procesos el speedup fue 3.26, 3.08 y 1.91, y con 8 procesos 3.37, 3.20 y 2.67: la ejecución 3 fue más lenta en las configuraciones paralelas con tiempos de 4 a 5 ms, mientras que el secuencial fue igual (0.010 s). La causa no se pudo confirmar. Una posible explicación es una perturbación temporal del equipo (otros programas o gestión de energía del procesador) que, con generaciones de pocos milisegundos, pesa mucho. Por eso las diferencias pequeñas entre configuraciones en 100x100 no deben interpretarse, y es la razón para repetir el benchmark.

Con las tres ejecuciones la conclusión es más sólida: el máximo se alcanza siempre entre 16 y 20 procesos, en 500x500 y 1000x1000 el speedup máximo de cada ejecución está entre 5.4 y 6.0, y 40 procesos es siempre peor que 16. Las curvas de 500x500 y 1000x1000 casi coinciden en todas las ejecuciones.

9. Conclusiones

- La paralelización por bloques de filas con MPI es correcta (produce exactamente la misma evolución que la versión secuencial en las tres ejecuciones) y efectiva en matrices medianas y grandes: con 8 procesos el speedup mediano es de 4.24 en 500x500 y de 4.33 en 1000x1000, y con 16 llega a 5.65 y 5.43.
- El beneficio no escala linealmente con el número de procesos. La eficiencia (speedup dividido entre procesos) en 500x500 y 1000x1000 es de 90 % a 92 % con 2 procesos, de 82 % a 83 % con 4, de 53 % a 54 % con 8 y de 34 % a 35 % con 16. En este equipo el rango útil es de 4 a 16 procesos: usar los 20 hilos no mejora respecto a 16 y con 40 empeora.
- En matrices pequeñas (100x100) el paralelismo aporta menos y es menos fiable: el máximo mediano es 4.52 con 16 procesos, el resultado varía mucho entre ejecuciones (con 4 procesos, de 1.91 a 3.26) y con 40 procesos el speedup cae a 1.39. Con generaciones de pocos milisegundos, el costo de coordinar procesos domina.
- Pasar de 500x500 a 1000x1000 no mejora el speedup (5.65 y 5.43 con 16 procesos): la ganancia por tamaño se agota cuando el trabajo por proceso ya es suficientemente grande frente al costo fijo de coordinar.
- La limitación principal de este diseño es la comunicación y no el cálculo: enviar la matriz completa a todos los procesos en cada generación, serializada con pickle, deja una fracción de 10 % a 15 % del tiempo sin paralelizar. Es la explicación más coherente con los datos, aunque no se midió directamente.
- El speedup se mide contra una versión secuencial deliberadamente simple, en Python puro. Con una versión secuencial más rápida (por ejemplo, vectorizada con NumPy) el cálculo pesaría menos y el costo de comunicación se notaría aún más.
- Las mediciones se repitieron tres veces. En 500x500 y 1000x1000 los resultados son estables (el speedup máximo de cada ejecución varía entre 5.4 y 6.0); 100x100 varió mucho más y sus diferencias pequeñas no deben interpretarse. Reportar la mediana de varias ejecuciones hace los resultados menos sensibles a una medición anómala.

10. Limitaciones y mejoras

- **Reducir la comunicación:** repartir con scatter solo el bloque de cada proceso, junto con una fila de borde superior y una inferior, e intercambiar esas filas halo con los vecinos (Sendrecv) en cada generación, en lugar de enviar la matriz completa a todos.
- **Evitar la serialización:** usar matrices NumPy y las funciones de MPI para buffers (Bcast, Gatherv, con mayúscula) que no usan pickle, y no recolectar la matriz en el proceso 0 en cada generación cuando no se está dibujando.
- **Más verificación:** la comprobación de correctitud solo se hizo con 60x60 y 4 procesos. Falta probar casos donde n no es divisible entre p y otros números de procesos.
- **Mediciones más sólidas:** repetir el benchmark más veces (aquí se hicieron tres y se reporta la mediana), reportar también la dispersión, cerrar otros programas durante la prueba, imprimir los tiempos con más decimales y usar más generaciones en 100x100 para reducir el ruido.
- **Medir la comunicación** de forma directa (tiempo de bcast, de cálculo y de gather por separado) para confirmar la explicación de la sección 8.2.
- **Probar más de una máquina:** al ser MPI un modelo de memoria distribuida, el mismo código podría ejecutarse en varios equipos, donde el costo de la comunicación sería todavía más importante.